

Cross-chain liquidity from theory to application: the openBDK reference

openBDK inherits the AggLayer Unified Bridge — a cross-chain liquidity primitive engineered from the wreckage of \$2.8 billion in bridge failures. Unlike traditional bridges that trust small validator committees or wrap tokens into exploitable honeypots, the Unified Bridge uses pessimistic proofs to cryptographically verify that no connected chain can withdraw more than it deposited. (Agglayer +2) This document covers the full stack: the theory of cross-chain liquidity, a forensic analysis of every major bridge failure, the architecture that prevents them, and working Solidity code for building on top of it.

openBDK is derived from Polygon's AggLayer architecture and uses the EVM as its execution environment. (Agglayer) The LAIR3-BDK project (github.com/LAIR3) extends Polygon CDK's Kurtosis deployment framework into a modular Blockchain Development Kit, (github) now evolving toward openBDK v7 with Relayer-Validator consensus layered on top of AggLayer's settlement guarantees. Every design decision in openBDK traces back to a specific class of bridge failure that it refuses to repeat.

Part 1 — The theory of cross-chain liquidity

Why multi-chain architectures emerged from the scalability trilemma

The blockchain trilemma — formalized mathematically in a 2024 *Applied Sciences* paper (MDPI, 15(1):19) — proves that the product of decentralization, security, and scalability is bounded by a constant. No monolithic chain optimizes all three simultaneously. This constraint drives the modular thesis: execution moves to Layer 2 rollups and app-chains, settlement remains on Ethereum L1, and data availability distributes across specialized layers like Celestia and EigenDA.

The result is a proliferation of independent execution environments. L2BEAT tracks over **\$31.4 billion** in Total Value Secured across Ethereum L2s as of April 2026. (L2BEAT) Arbitrum One holds roughly 44% of L2 TVL, Base captures 33%, and OP Mainnet accounts for 6%. (Cryptopolitan) Each chain runs its own state machine, its own sequencer, and its own liquidity pools — fragmenting what should be a unified economic system into dozens of siloed markets.

Liquidity fragmentation manifests in three forms. Cross-chain fragmentation scatters the same asset across multiple networks. Intra-chain fragmentation distributes liquidity across competing protocols on a single chain. And wrapped token fragmentation — perhaps the most insidious — produces multiple non-interchangeable representations of the same underlying asset, one per bridge provider. (Kima) A 2024 *Frontiers in Blockchain* paper captures the paradox: each new

bridge brings a new collection of bridge-wrapped tokens, exacerbating the problem bridges were built to solve.

Trust assumptions across the bridge spectrum

All practical cross-chain communication requires some trusted component. (arXiv) Zamyatin et al. (2019, IACR ePrint 2019/1128) proved this formally: there exists no asynchronous cross-chain communication protocol tolerant of misbehaving nodes without a trusted third party. The proof reduces cross-chain communication to the Fair Exchange problem (Pagnia & Gaertner, 1999), known to be impossible without a TTP. The question is not "trusted vs. trustless" but **how much trust, and of what kind**.

Bridge architectures sit on a trust-minimization spectrum:

Level	Model	Trust assumption	Examples
1	Centralized custody	Single entity	Exchange bridges
2	Multisig / MPC	M-of-N committee honesty	Ronin (5/9), Harmony (2/5)
3	Optimistic	At least 1 honest watcher	Across, Nomad (pre-hack)
4	Light client	Source chain consensus	Cosmos IBC, Rainbow Bridge
5	ZK validity proof	Cryptographic soundness	zkBridge, Polyhedra
6	Native shared bridge	Unified settlement + ZK accounting	AggLayer Unified Bridge

Every bridge hack in recorded history exploited a design at levels 1–3. No bridge using on-chain light client verification or ZK validity proofs has suffered a comparable exploit. (Berkeley) The industry trajectory is unambiguous: away from human trust assumptions, toward mathematical guarantees.

Asset transfer models and their security implications

Lock-and-mint locks native tokens on a source chain and mints wrapped equivalents on the destination. (Polygon +2) (Altrady) This is the most common model and the most dangerous: it concentrates maximum value in a single smart contract, creating a honeypot. (Elliptic) The Ronin bridge held over \$600M, Wormhole over \$320M. A single exploit drains the entire pool.

Burn-and-mint destroys tokens on the source chain and mints native equivalents on the destination. (FinanceFeeds) Circle's Cross-Chain Transfer Protocol (CCTP) implements this for USDC — eliminating wrapped token risk and custodial pools. (StablecoinInsider) The risk shifts entirely to the verification layer.

Native issuance (the AggLayer model) avoids wrapping entirely. Tokens bridged through the Unified Bridge retain native fungibility across all connected chains. [\(Agglayer\)](#) There is no wrapped-vs-native distinction because all chains share a single bridge escrow on L1 with cryptographic accounting per chain.

Cryptographic primitives underpinning bridge security

Merkle proofs compress datasets into a single root hash with $O(\log n)$ inclusion proofs. In bridges, they verify that a cross-chain transaction occurred on the source chain by providing a path from the transaction leaf to the block's state root. The AggLayer uses 32-depth Sparse Merkle Trees for exit commitments, balance tracking, and nullifier management.

Zero-knowledge proofs allow proving computational statements without revealing underlying data. The AggLayer's pessimistic proofs use **STARK** proofs (generated via Plonky3) [\(Agglayer\)](#) wrapped in a **Groth16** SNARK for on-chain verification on Ethereum. [\(GitHub\)](#) The Berkeley RDI *zkBridge* paper (2022) demonstrated the first practical ZK-SNARK bridge achieving trustless verification without committee assumptions. [\(Berkeley\)](#)

Threshold signatures and MPC distribute signing authority across multiple parties. The security assumption — that fewer than t parties are compromised simultaneously — has been violated repeatedly in practice. Ronin's 5-of-9 scheme fell to social engineering. [\(Cointelegraph\)](#) [\(The Record\)](#) Harmony's 2-of-5 fell even faster. [\(CNBC\)](#) Multichain's MPC keys were lost entirely when its CEO was arrested. [\(Chainalysis\)](#)

Part 2 — The graveyard of cross-chain failures

\$2.8 billion in bridge exploits and what each one teaches

openBDK is designed from the forensic record of every major bridge failure. The table below summarizes the damage; the analysis that follows maps each failure to the architectural decision that prevents it.

Bridge	Date	Amount lost	Root cause	Attribution
Ronin	Mar 23, 2022	\$624M	5/9 validator key compromise	Lazarus Group (FBI-confirmed)
Poly Network	Aug 10, 2021	\$611M	Access control bug in cross-chain manager	Unknown ("Mr. White Hat")
BNB Bridge	Oct 6, 2022	\$570M minted (~\$137M exfiltrated)	IAVL Merkle proof forgery	Unknown
Wormhole	Feb 2, 2022	\$320M	Signature verification bypass on Solana	Unknown
Nomad	Aug 1, 2022	\$190M	Trusted root set to zero hash on upgrade	Crowdsourced (hundreds of copycats)
Harmony Horizon	Jun 24, 2022	\$100M	2/5 multisig compromise	Lazarus Group (FBI-confirmed)
Multichain	Jul 7, 2023	\$126M+	MPC key loss (CEO arrested)	Linked to CEO arrest
Qubit	Jan 27, 2022	\$80M	Legacy deposit function + missing SafeERC20	Unknown
Meter.io	Feb 5, 2022	\$4.4M	Deposit validation bypass	Unknown
ChainSwap	Jul 10, 2021	\$8M	Quota validation flaw	Unknown
THORChain	Jun-Jul 2021 (×3)	\$13M	Bifrost routing logic bugs	Partially white-hat

Total documented bridge losses: ~\$2.65 billion. North Korea's Lazarus Group alone accounts for \$724 million confirmed (Ronin + Harmony).

Ronin Bridge — the \$624M social engineering masterclass

On March 23, 2022, attackers compromised 5 of the 9 validator keys securing the Ronin Bridge. (Cointelegraph) Four keys belonged to Sky Mavis (a single entity); the fifth was an Axie DAO validator still authorized due to an improperly terminated temporary allowlist from November 2021. The attackers — later confirmed as North Korea's Lazarus Group by the FBI and OFAC —

used a fake job offer containing malware to compromise Sky Mavis employees. [Bleeping Computer](#) With five keys, they authorized two fraudulent withdrawals draining 173,600 ETH and 25.5M USDC. [Cointelegraph](#) **The exploit went undetected for six days.**

AggLayer prevention: The Unified Bridge has no validator multisig controlling withdrawals. Cross-chain claims require Merkle proof verification against cryptographically committed exit roots. [github](#) Even if every validator on a connected chain is compromised, the pessimistic proof ensures withdrawals cannot exceed deposits. [Agglayer](#)

Wormhole — forging guardian signatures with a deprecated syscall

On February 2, 2022, an attacker exploited Wormhole's Solana-side signature verification. The [verify_signatures](#) function delegated to a deprecated Solana [Secp256k1](#) system program that did not properly validate the sysvar account address. The attacker substituted a counterfeit sysvar, bypassing signature verification entirely, and minted 120,000 wETH (\$320M) on Solana without depositing corresponding ETH. Jump Crypto replaced the full amount from its own funds within 24 hours.

AggLayer prevention: Verification in the Unified Bridge is on-chain Merkle proof validation against committed exit roots, not external signature verification. The claim function ([claimAsset](#)) verifies two 32-depth SMT proofs — one against the Local Exit Root and one against the Rollup Exit Root ([polygon](#)) — making signature forgery irrelevant. [github](#)

Nomad — the upgrade bug that let anyone drain the bridge

During a routine smart contract upgrade on June 21, 2022, Nomad's [initialize](#) function set the zero hash ([0x00](#)) as a trusted root. Six weeks later, on August 1, attackers discovered that [acceptableRoot](#) returned [true](#) for any message with a zero hash. The first attacker drained funds; hundreds of copycats replicated the exploit by simply copy-pasting the transaction and replacing the recipient address. **\$190M was drained in a crowdsourced mass exploit.**

[Google Cloud](#)

AggLayer prevention: The Global Exit Root is computed deterministically as [keccak256\(rollupExitRoot, mainnetExitRoot\)](#) [GitHub](#) and verified against state committed on Ethereum L1. [github](#) There is no initialization step that could set a trusted root to zero — the exit tree is append-only, and roots are derived from actual bridge operations.

Harmony Horizon — why 2-of-5 is not a security model

The Harmony Horizon Bridge required only 2-of-5 signatures to authorize withdrawals. [CNBC](#) On June 24, 2022, the Lazarus Group compromised two private keys — sufficient to drain the entire \$100M bridge. Concerns about the bridge's dangerous centralization had been raised publicly months before the hack.

AggLayer prevention: Withdrawals from the Unified Bridge are not authorized by any multisig. They require a valid Merkle inclusion proof against an exit root that was committed to Ethereum via a pessimistic proof. The number of signers is irrelevant — **security derives from cryptographic proof verification, not committee size.**

Five systematic failure patterns the industry refused to learn

Pattern 1 — Small multisigs concentrate trust dangerously. A 2-of-5 scheme means an attacker needs only two keys. A 5-of-9 scheme controlled by a single entity is effectively 1-of-1. The cost to compromise a small validator set is typically far lower than the value secured.

Pattern 2 — Lock-and-mint creates honeypots. Unlike DEX liquidity pools distributed across many contracts, bridge contracts centralize hundreds of millions in a single smart contract.

(Elliptic) The Ronin bridge was a \$600M honeypot; the Wormhole bridge a \$320M honeypot.

Pattern 3 — Optimistic bridges trade security for speed. Seven-day challenge windows exist because they must provide sufficient time for fraud detection. (Changelly) (Altrady) Shorter windows risk censorship of fraud proofs. Longer windows destroy capital efficiency. Neither option is acceptable at scale.

Pattern 4 — External validators inherit none of L1's security. Ethereum's validator set comprises hundreds of thousands of nodes with billions staked. Bridge validator sets have 4-20 nodes with minimal economic stake. The security gap is orders of magnitude.

Pattern 5 — Single points of failure cascade. Multichain's CEO was arrested, his hardware confiscated, and the MPC keys lost. \$126M vanished because the entire protocol's security rested on one person's physical possession of keys.

Part 3 — The Unified Bridge architecture that doesn't fail

A single bridge contract secured by mathematical proof

The Unified Bridge is deployed as (PolygonZkEVMBridgeV2) on both Ethereum L1 and every connected L2 chain. (polygon +3) On Ethereum mainnet, the bridge contract sits at address (0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe) (UNPKG) — verified on-chain as a (TransparentUpgradeableProxy) (EIP-1967) with implementation (AggLayerBridge) at (0x66E0120e3c965552a89AcC37b03f762624baC5Ad). Created on March 24, 2023, it holds the unified escrow for all AggLayer-connected chains.

Two companion contracts complete the L1 architecture:

- **PolygonRollupManager** ((0x5132A183E9F3CB7C848b0AAC5Ae0c4f0491B7aB2)) (UNPKG) — Aggregates all L2 Local Exit Roots into a Rollup Exit Tree; (GitHub) verifies pessimistic

proofs and validity proofs; maps chain IDs to rollup indices. ([github](#))

- **PolygonZkEVMGlobalExitRootV2** ([0x580bda1e7A0CFAe92Fa7F6c20A3794F169CE3CFb](#)) ([GitHub](#)) ([UNPKG](#)) — Computes (`globalExitRoot = keccak256(rollupExitRoot, mainnetExitRoot)`) and appends each new GER to the L1 Info Tree. ([github](#))

Source code for all contracts lives in the ([agglayer/agglayer-contracts](#)) repository (formerly [0xPolygonHermes/zkevm-contracts](#)), compiled with (`pragma solidity 0.8.20`). ([GitHub](#))

The hierarchical Sparse Merkle Tree architecture

The Unified Bridge maintains a hierarchical tree structure:

On-chain bridge trees (deployed in smart contracts):

The **Local Exit Tree** (LET) is a 32-depth binary Sparse Merkle Tree maintained per chain.

([GitHub](#)) ([GitHub](#)) Each leaf is a hash of a (`bridgeAsset`) or (`bridgeMessage`) operation. Every chain — L1 and each L2 — maintains its own LET in its local (`PolygonZkEVMBridgeV2`) deployment.

([Agglayer](#)) The root is called the Local Exit Root (LER), and leaves are indexed by (`depositCount`).

([github](#))

The **Rollup Exit Tree** (RET) exists only on L1 in (`PolygonRollupManager`). Its leaves are the Local Exit Roots of all connected L2 chains. When an L2 submits its LER, the RET updates and triggers a Global Exit Root update. ([GitHub](#)) ([github](#))

The **Global Exit Root** combines both: (`GER = keccak256(rollupExitRoot, mainnetExitRoot)`). Each new GER is appended to the **L1 Info Tree** ([GitHub](#)) — another 32-depth SMT that L2 chains sync to verify claims. ([github](#))

Pessimistic proof state trees (off-chain, per chain):

Each chain's (`LocalNetworkState`) comprises three additional Sparse Merkle Trees used by the pessimistic proof system: ([Agglayer](#)) ([Agglayer](#))

```
rust

pub struct LocalNetworkState {
    pub exit_tree: LocalExitTree<Keccak256Hasher>,    // depth 32
    pub balance_tree: LocalBalanceTree<Keccak256Hasher>, // depth 192
    pub nullifier_tree: NullifierTree<Keccak256Hasher>, // depth 64
}
```

The **Local Balance Tree** (192-bit depth) tracks per-token balances for bridge accounting. Keys encode (`TokenInfo`) as 32 bits of (`origin_network`) concatenated with 160 bits of (`origin_token_address`). ([GitHub](#)) Outbound bridging decreases the balance; inbound claiming

increases it. [Agglayer](#) **The core invariant: for every token on every chain, total withdrawable $\leq$ total deposited.**

The **Nullifier Tree** (64-bit depth) prevents double-claiming. Keys encode 32 bits of origin network ID concatenated with 32 bits of deposit index. [Agglayer](#) Once a bridge exit is claimed, its global index is appended to the Nullifier Tree and can never be claimed again. [GitHub](#) [Agglayer](#)

Why "pessimistic" — assuming every chain is malicious

The pessimistic proof system assumes all connected chains are potentially compromised. Unlike optimistic approaches that assume honesty unless challenged, pessimistic proofs [Agglayer](#) **cryptographically verify bridge accounting before settlement.**

The proof attests to two statements:

1. **Balance conservation:** Every withdrawal claim is backed by deposits. For each token on each chain: `total_withdrawable  $\leq$  total_deposited`.
2. **Nullifier integrity:** No double-claims. Each bridge exit is claimed at most once. [Agglayer](#)

Critically, pessimistic proofs do **not** verify L2 state transition correctness — that remains the responsibility of each chain's own proof system (ZK validity proof, fraud proof, or trusted sequencer signature). The pessimistic proof is agnostic to consensus mechanism. [GitHub](#) A compromised chain can harm its own users but **cannot drain the shared bridge or affect other chains.** [Agglayer](#)

The proof is written as standard Rust [Succinct](#) (~1,000 lines by two Polygon engineers) and proven inside **SP1 zkVM** (Succinct Labs), which compiles Rust to RISC-V and generates **Plonky3 STARK** proofs. [Agglayer +2](#) Over 75% of computation is Keccak hashing for Merkle proof verification; [GitHub](#) SP1's Keccak precompile provides order-of-magnitude speedup. Production proofs use the Succinct Prover Network; [GitHub](#) [GitHub](#) the output STARK can be wrapped into a **Groth16** proof for cheap EVM verification.

```
rust
```



```
// Pessimistic proof SP1 program entry point
// Source: agglayer/crates/pessimistic-proof-program/src/main.rs
use pessimistic_proof_core::{generate_pessimistic_proof, NetworkState, PessimisticPr
sp1_zkvm::entrypoint!(main);

pub fn main() {
    let initial_state = sp1_zkvm::io::read::<NetworkState>();
    let batch_header = sp1_zkvm::io::read::<MultiBatchHeader<Keccak256Hasher>>();
    let (outputs, _targets) = generate_pessimistic_proof(
        initial_state, &batch_header
    ).unwrap();
    let pp_inputs = PessimisticProofOutput::bincode_options()
        .serialize(&outputs).unwrap();
    sp1_zkvm::io::commit_slice(&pp_inputs);
}
```

Certificate settlement flow from chain to Ethereum

The AggLayer operates on an epoch-based settlement architecture:

1. **Bridge event collection:** The `BridgeSync` component on each CDK chain monitors `BridgeEvent` emissions from the local bridge contract.
2. **Certificate construction:** The `AggSender` builds a certificate containing bridge exits, imported bridge exits (claims), and the new Local Exit Root.
3. **Certificate submission:** The certificate is submitted to the AggLayer node via JSON-RPC.
4. **Pessimistic proof generation:** The `Certifier` component applies the certificate to the existing `LocalNetworkState`, runs the proof logic natively in Rust for correctness verification, then generates the SP1 ZK proof. [deepwiki](#) [GitHub](#)
5. **Epoch packing:** The `EpochPacker` aggregates proven certificates across chains for the current epoch.
6. **L1 settlement:** The `SettlementClient` submits the proof to `PolygonRollupManager` on Ethereum, which verifies it and updates the Rollup Exit Root. [deepwiki](#)
7. **Global Exit Root update:** `PolygonZkEVMGlobalExitRootV2` computes the new `globalExitRoot` and appends it to the L1 Info Tree. [github](#)
8. **L2 synchronization:** Destination chains sync the latest GER via `PolygonZkEVMGlobalExitRootL2` and can process claims. [GitHub](#) [github](#)

Certificate lifecycle states progress through: `Pending → Proven → Candidate → Settled`. A certificate that fails validation enters `InError` and does not progress — the chain's previous state remains authoritative.

Security properties that prevent every historical failure class

A compromised chain can only drain its own deposited funds. The Local Balance Tree enforces per-chain, per-token accounting. Chain A's balance is independent of Chain B's deposits. A malicious prover on Chain A cannot forge a pessimistic proof that claims Chain B's tokens — the proof verifies against Chain A's own balance tree.

No validator committee to compromise. Claims require on-chain Merkle proof verification, not multisig approval. The Ronin attack vector (social engineering 5 keys), the Harmony attack vector (compromising 2 keys), and the Multichain attack vector (seizing one person's hardware) are all architecturally impossible.

No wrapped token honeypots. All assets are held in a single unified escrow on Ethereum, with per-chain accounting enforced by the pessimistic proof. `Polygon` There is no separate "bridge contract per chain" to accumulate into a concentrated target.

No trusted root initialization. The Global Exit Root is deterministically computed from actual bridge operations. The Nomad attack (zero hash trusted root) cannot occur because roots are derived, not configured.

Part 4 — Building on the Unified Bridge

The IPolygonZkEVMBridgeV2 interface

The Unified Bridge exposes four primary functions. The complete interface, compiled against `pragma solidity ^0.8.20`:

```
solidity
```

```

// SPDX-License-Identifier: AGPL-3.0
pragma solidity ^0.8.20;

interface IPolygonZkEVMBridgeV2 {
    /// @notice Bridge assets from one network to another
    /// @param destinationNetwork Network ID (0=Ethereum, 1=zkEVM, ...)
    /// @param destinationAddress Recipient on destination chain
    /// @param amount Token amount in wei
    /// @param token Token address (address(0) for native ETH)
    /// @param forceUpdateGlobalExitRoot Immediately update GER
    /// @param permitData Optional ERC-2612 permit data
    function bridgeAsset(
        uint32 destinationNetwork,
        address destinationAddress,
        uint256 amount,
        address token,
        bool forceUpdateGlobalExitRoot,
        bytes calldata permitData
    ) external payable;

    /// @notice Bridge an arbitrary message to another network
    function bridgeMessage(
        uint32 destinationNetwork,
        address destinationAddress,
        bool forceUpdateGlobalExitRoot,
        bytes calldata metadata
    ) external payable;

    /// @notice Claim bridged assets on destination chain
    /// @param smtProofLocalExitRoot Merkle proof against local exit root
    /// @param smtProofRollupExitRoot Merkle proof against rollup exit root
    /// @param globalIndex 256-bit: [191 unused][1 mainnetFlag][32 rollupIdx][32 loc
    function claimAsset(
        bytes32[32] calldata smtProofLocalExitRoot,
        bytes32[32] calldata smtProofRollupExitRoot,
        uint256 globalIndex,
        bytes32 mainnetExitRoot,
        bytes32 rollupExitRoot,
        uint32 originNetwork,
        address originTokenAddress,
        uint32 destinationNetwork,
        address destinationAddress,
        uint256 amount,

```

```

        bytes calldata metadata
    ) external;

/// @notice Claim a bridged message on destination chain
function claimMessage(
    bytes32[32] calldata smtProofLocalExitRoot,
    bytes32[32] calldata smtProofRollupExitRoot,
    uint256 globalIndex,
    bytes32 mainnetExitRoot,
    bytes32 rollupExitRoot,
    uint32 originNetwork,
    address originAddress,
    uint32 destinationNetwork,
    address destinationAddress,
    uint256 amount,
    bytes calldata metadata
) external;

/// @notice Update global exit root with latest deposit
function updateGlobalExitRoot() external;

/// @notice Check if a specific bridge exit has been claimed
function isClaimed(
    uint32 leafIndex,
    uint32 sourceBridgeNetwork
) external view returns (bool);

event BridgeEvent(
    uint8 leafType,
    uint32 originNetwork,
    address originAddress,
    uint32 destinationNetwork,
    address destinationAddress,
    uint256 amount,
    bytes metadata,
    uint32 depositCount
);

event ClaimEvent(
    uint256 globalIndex,
    uint32 originNetwork,
    address originAddress,
    address destinationAddress,
    uint256 amount

```

```
    );  
}
```

Message receivers must implement:

```
solidity  
  
// SPDX-License-Identifier: AGPL-3.0  
pragma solidity ^0.8.20;  
  
interface IBridgeMessageReceiver {  
    function onMessageReceived(  
        address originAddress,  
        uint32 originNetwork,  
        bytes memory data  
    ) external payable;  
}
```

[GitHub](#)

[Polygon](#)

Example: cross-chain token transfer contract

```
solidity
```

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

interface IPolygonZkEVMBridgeV2 {
    function bridgeAsset(
        uint32 destinationNetwork,
        address destinationAddress,
        uint256 amount,
        address token,
        bool forceUpdateGlobalExitRoot,
        bytes calldata permitData
    ) external payable;
}

/// @title CrossChainVault
/// @notice Bridges ERC-20 tokens or native ETH via the Unified Bridge
/// @dev Deploy on any AggLayer-connected chain
contract CrossChainVault {
    using SafeERC20 for IERC20;

    IPolygonZkEVMBridgeV2 public immutable bridge;

    event TokensBridged(
        address indexed sender,
        uint32 destinationNetwork,
        address token,
        uint256 amount
    );

    constructor(address _bridge) {
        bridge = IPolygonZkEVMBridgeV2(_bridge);
    }

    /// @notice Bridge ERC-20 tokens to a destination network
    function bridgeTokens(
        uint32 destinationNetwork,
        address destinationAddress,
        address token,
        uint256 amount
    ) external {

```

```

IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
IERC20(token).forceApprove(address(bridge), amount);

bridge.bridgeAsset(
    destinationNetwork,
    destinationAddress,
    amount,
    token,
    true,    // force GER update for faster finality
    ""      // no permit data
);

emit TokensBridged(msg.sender, destinationNetwork, token, amount);
}

/// @notice Bridge native ETH to a destination network
function bridgeETH(
    uint32 destinationNetwork,
    address destinationAddress
) external payable {
    require(msg.value > 0, "No ETH sent");

    bridge.bridgeAsset{value: msg.value}(
        destinationNetwork,
        destinationAddress,
        msg.value,
        address(0), // address(0) = native ETH
        true,
        ""
    );

    emit TokensBridged(
        msg.sender, destinationNetwork, address(0), msg.value
    );
}
}

```

Example: cross-chain messaging with receiver authentication

solidity

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface IPolygonZkEVMBridgeV2 {
    function bridgeMessage(
        uint32 destinationNetwork,
        address destinationAddress,
        bool forceUpdateGlobalExitRoot,
        bytes calldata metadata
    ) external payable;
}

interface IBridgeMessageReceiver {
    function onMessageReceived(
        address originAddress,
        uint32 originNetwork,
        bytes memory data
    ) external payable;
}

/// @title CrossChainMessenger – Sender
/// @notice Deploy on any source chain
contract CrossChainMessenger {
    IPolygonZkEVMBridgeV2 public immutable bridge;

    constructor(address _bridge) {
        bridge = IPolygonZkEVMBridgeV2(_bridge);
    }

    function sendMessage(
        uint32 destinationNetwork,
        address receiver,
        bytes calldata payload
    ) external {
        bridge.bridgeMessage(
            destinationNetwork,
            receiver,
            true,
            payload
        );
    }
}

```



```

/// @title CrossChainReceiver – Authenticated message handler
/// @notice Deploy on destination chain. Only processes messages
///         from a trusted sender on a trusted origin network.
contract CrossChainReceiver is IBridgeMessageReceiver {
    address public immutable bridgeAddress;
    address public immutable trustedSender;
    uint32 public immutable trustedOriginNetwork;

    event MessageProcessed(
        address indexed originAddress,
        uint32 originNetwork,
        bytes data
    );

    constructor(
        address _bridge,
        address _trustedSender,
        uint32 _originNetwork
    ) {
        bridgeAddress = _bridge;
        trustedSender = _trustedSender;
        trustedOriginNetwork = _originNetwork;
    }

    function onMessageReceived(
        address originAddress,
        uint32 originNetwork,
        bytes memory data
    ) external payable override {
        require(msg.sender == bridgeAddress, "Only bridge");
        require(originAddress == trustedSender, "Untrusted sender");
        require(
            originNetwork == trustedOriginNetwork,
            "Wrong origin network"
        );

        _processMessage(data);
        emit MessageProcessed(originAddress, originNetwork, data);
    }

    function _processMessage(bytes memory data) internal virtual {
        // Override in derived contracts to implement business logic
    }
}

```

}

}

Example: cross-chain governance voting

solidity

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface IPolygonZkEVMBridgeV2 {
    function bridgeMessage(
        uint32 destinationNetwork,
        address destinationAddress,
        bool forceUpdateGlobalExitRoot,
        bytes calldata metadata
    ) external payable;
}

interface IBridgeMessageReceiver {
    function onMessageReceived(
        address originAddress,
        uint32 originNetwork,
        bytes memory data
    ) external payable;
}

/// @title VoteBridge - Submit votes from any connected L2
/// @notice Deploy one instance per L2 chain
contract VoteBridge {
    IPolygonZkEVMBridgeV2 public immutable bridge;
    address public immutable governanceReceiver;
    uint32 public immutable governanceNetwork;

    constructor(
        address _bridge,
        address _receiver,
        uint32 _govNetwork
    ) {
        bridge = IPolygonZkEVMBridgeV2(_bridge);
        governanceReceiver = _receiver;
        governanceNetwork = _govNetwork;
    }

    function castVote(
        uint256 proposalId,
        bool support,
        uint256 votingPower
    ) external {
        bytes memory payload = abi.encode(

```

```

        msg.sender,
        proposalId,
        support,
        votingPower,
        block.timestamp
    );

    bridge.bridgeMessage(
        governanceNetwork,
        governanceReceiver,
        true,
        payload
    );
}
}

/// @title GovernanceTally – Aggregate votes from all chains
/// @notice Deploy on the governance home chain
contract GovernanceTally is IBridgeMessageReceiver {
    address public immutable bridgeAddress;

    struct Proposal {
        uint256 forVotes;
        uint256 againstVotes;
        bool    executed;
    }

    mapping(uint256 => Proposal) public proposals;
    mapping(uint256 => mapping(address => bool)) public hasVoted;
    mapping(uint32  => address) public trustedVoteBridges;

    event VoteReceived(
        address voter,
        uint256 proposalId,
        bool support,
        uint256 power,
        uint32 fromNetwork
    );

    constructor(address _bridge) {
        bridgeAddress = _bridge;
    }

    function setTrustedBridge(

```

```

        uint32 networkId,
        address sender
    ) external {
        // Production: add Ownable or AccessControl
        trustedVoteBridges[networkId] = sender;
    }

    function onMessageReceived(
        address originAddress,
        uint32 originNetwork,
        bytes memory data
    ) external payable override {
        require(msg.sender == bridgeAddress, "Only bridge");
        require(
            trustedVoteBridges[originNetwork] == originAddress,
            "Untrusted source"
        );

        (
            address voter,
            uint256 proposalId,
            bool support,
            uint256 power,
        ) = abi.decode(data, (address, uint256, bool, uint256, uint256));

        require(!hasVoted[proposalId][voter], "Already voted");
        hasVoted[proposalId][voter] = true;

        if (support) {
            proposals[proposalId].forVotes += power;
        } else {
            proposals[proposalId].againstVotes += power;
        }

        emit VoteReceived(voter, proposalId, support, power, originNetwork);
    }
}

```

Foundry test suite for cross-chain operations

solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";

interface IPolygonZkEVMBridgeV2 {
    function bridgeAsset(
        uint32 destinationNetwork,
        address destinationAddress,
        uint256 amount,
        address token,
        bool forceUpdateGlobalExitRoot,
        bytes calldata permitData
    ) external payable;

    function bridgeMessage(
        uint32 destinationNetwork,
        address destinationAddress,
        bool forceUpdateGlobalExitRoot,
        bytes calldata metadata
    ) external payable;

    function isClaimed(
        uint32 leafIndex,
        uint32 sourceBridgeNetwork
    ) external view returns (bool);
}

contract UnifiedBridgeTest is Test {
    // Mainnet Unified Bridge address (same on L1 and L2)
    address constant BRIDGE = 0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe;

    uint256 ethereumFork;
    uint256 zkevmFork;

    function setUp() public {
        ethereumFork = vm.createFork(
            vm.envString("ETHEREUM_RPC_URL")
        );
        zkevmFork = vm.createFork(
            vm.envString("POLYGON_ZKEVM_RPC_URL")
        );
    }
}
```

```

function test_BridgeExists_OnBothForks() public {
    vm.selectFork(ethereumFork);
    assertGt(BRIDGE.code.length, 0, "Bridge missing on L1");

    vm.selectFork(zkevmFork);
    assertGt(BRIDGE.code.length, 0, "Bridge missing on L2");
}

function test_BridgeAsset_ETH_L1toL2() public {
    vm.selectFork(ethereumFork);

    address user = makeAddr("user");
    vm.deal(user, 10 ether);

    uint256 bridgeBalBefore = BRIDGE.balance;

    vm.prank(user);
    IPolygonZkEVMBridgeV2(BRIDGE).bridgeAsset{value: 1 ether}(
        1,                // destinationNetwork: zkEVM
        user,             // destinationAddress
        1 ether,          // amount
        address(0),       // native ETH
        true,             // forceUpdateGlobalExitRoot
        ""               // no permit
    );

    assertEq(
        BRIDGE.balance,
        bridgeBalBefore + 1 ether,
        "ETH not locked in bridge"
    );
}

function test_BridgeMessage_L1toL2() public {
    vm.selectFork(ethereumFork);

    address sender = makeAddr("sender");
    vm.deal(sender, 1 ether);

    bytes memory message = abi.encode(
        "openBDK cross-chain ping", uint256(block.number)
    );
}

```

```

    vm.prank(sender);
    IPolygonZkEVMBridgeV2(BRIDGE).bridgeMessage(
        1, // destinationNetwork
        makeAddr("receiver"), // destinationAddress
        true, // forceUpdateGlobalExitRoot
        message // metadata
    );
}

function test_UnclaimedDeposit() public {
    vm.selectFork(ethereumFork);
    // A non-existent deposit should not be claimed
    bool claimed = IPolygonZkEVMBridgeV2(BRIDGE).isClaimed(
        99999, 1
    );
    assertFalse(claimed, "Ghost deposit marked as claimed");
}
}

```

Foundry commands for multi-chain development:

```

bash

# Install dependencies
forge install OpenZeppelin/openzeppelin-contracts

# Run cross-chain fork tests
ETHEREUM_RPC_URL="https://eth.llamarpc.com" \
POLYGON_ZKEVM_RPC_URL="https://zkevm-rpc.com" \
forge test --match-contract UnifiedBridgeTest -vvv

# Deploy to a CDK chain
forge script script/Deploy.s.sol \
    --rpc-url $CDK_RPC_URL \
    --broadcast \
    --verify

# Fork test at a specific block
forge test --fork-url $ETHEREUM_RPC_URL --fork-block-number 19000000

```

Bridge-and-Call: atomic cross-chain contract invocation

The `lxly-bridge-and-call` extension (github.com/agglayer/lxly-bridge-and-call) enables atomically bridging assets **and** executing a contract call on the destination chain in a single

operation:

```
solidity

function bridgeAndCall(
    address token,
    uint256 amount,
    uint32 destinationNetwork,
    address callAddress,      // contract to call on destination
    address fallbackAddress,  // receives assets if call fails
    bytes calldata callData,  // encoded function call
    bool forceUpdateGlobalExitRoot
) external payable;
```

This uses a `JumpPoint` contract pattern on the destination chain. If the call succeeds, assets are forwarded to the target contract with the specified calldata. If it fails, assets go to the `fallbackAddress`. This enables one-click cross-chain DeFi operations — deposit-and-stake, swap-and-bridge, mint-and-transfer — without requiring users to submit transactions on both chains.

SDK and tooling ecosystem

- **AggLayer SDK** (`@agglayer/sdk`): TypeScript SDK with two modules — Multi-Bridge Routes (aggregates routes via the ARC API) and Native Bridge (direct on-chain interaction). Supports Local (AggSandbox) → Testnet (Sepolia) → Mainnet workflows.
- **lxly.js** (`github.com/OxPolygon/lxly.js`): JavaScript SDK wrapping `bridgeAsset`, `claimAsset`, `bridgeMessage`, `claimMessage` with type conversion and proof generation.
- **AggSandbox** (`github.com/agglayer/aggsandbox`): Local simulation spinning up L1 (port 8545), L2 zkEVM (port 8546), and bridge service. Install with `make install && make start`.
- **Polygon API**: Proof generation endpoint at `api-gateway.polygon.technology/api/v3/proof/mainnet/merkle-proof` (requires API key).

Important: AggLayer uses internal **Network IDs** (0=Ethereum, 1=zkEVM) distinct from standard Chain IDs (1=Ethereum, 1101=zkEVM). Use `PolygonRollupManager.chainIDToRollupID()` for conversion.

Part 5 — How openBDK inherits from AggLayer

From Kurtosis CDK to the Blockchain Development Kit

The LAIR3-BDK project (github.com/LAIR3) builds on Polygon's `kurtosis-cdk` framework — a Kurtosis package that deploys private, portable CDK devnets over Docker or Kubernetes. The BDK lineage progresses through versions: BDK4 (initial fork of kurtosis-cdk), BDK5 (TNT legacy devnet), BDK12op1 (OP Stack variant), and now toward openBDK v7.

Each BDK deployment provisions:

1. A local L1 chain with multi-client support via the `ethereum-package`
2. A local L2 chain using Polygon CDK components (sequencer, aggregator, prover, RPC, DAC)
3. The zkEVM bridge infrastructure for L1↔L2 asset bridging
4. The AggLayer for trustless cross-chain interoperability secured by ZK proofs
5. Additional services: transaction spammer, monitoring tools, permissionless nodes

openBDK extends this foundation with a **Relayer-Validator consensus layer** that wraps AggLayer's bridge settlement. The Relayer-Validator model introduces token-gated validator privilege within the AggLayer ecosystem — validators must hold a governance token to participate in block production and relaying, while the underlying bridge security remains enforced by pessimistic proofs regardless of validator behavior.

The security composition that makes this work

openBDK's security model is layered, not monolithic:

Layer 1 — Ethereum settlement. All cross-chain transactions ultimately settle on Ethereum L1. The Global Exit Root is committed to Ethereum's state, inheriting the full economic security of Ethereum's validator set (~\$100B+ staked).

Layer 2 — Pessimistic proof accounting. The AggLayer's pessimistic proof ensures that no chain — including an openBDK chain with compromised validators — can withdraw more than it deposited. This is enforced cryptographically, not by committee.

Layer 3 — Relayer-Validator consensus. openBDK's own consensus layer handles block production, transaction ordering, and relaying. Validators stake governance tokens and participate in a delegated consensus process. If validators collude, they can harm users *on their own chain* (censoring transactions, reordering for MEV) but they **cannot drain the bridge** or affect other AggLayer-connected chains.

This composition means openBDK inherits AggLayer's bridge safety guarantees unconditionally. The Relayer-Validator layer adds liveness and ordering properties on top of a

foundation that is already secure against the most catastrophic failure mode — bridge drainage.

How openBDK chains settle to Ethereum via AggLayer



Part 6 — Failure mode analysis for openBDK

Validator collusion (2/3 compromise) — contained by pessimistic proof

If 2/3 of openBDK's Relayer-Validators collude, they can produce arbitrary state transitions on the openBDK chain. They could censor users, front-run transactions, or propose blocks with invalid state roots. However, the pessimistic proof enforces that **the openBDK chain's withdrawals from the Unified Bridge never exceed its deposits**. A colluding validator set cannot fabricate deposits that didn't occur on L1. The damage is isolated to the openBDK chain itself — other AggLayer-connected chains are unaffected. This is the critical property that distinguishes AggLayer from every bridge that has been exploited: **compromise of a connected chain does not compromise the shared bridge**.

Relayer censorship — mitigated by L1 forced inclusion

If the openBDK sequencer or relayer censors transactions — refusing to include bridge operations in blocks — users' cross-chain operations would stall. The mitigation path is L1 forced inclusion: users submit transactions directly to the L1 bridge contract, bypassing the L2

sequencer entirely. The CDK codebase includes this functionality, though it is currently disabled in production. When activated, forced inclusion provides a censorship-resistance escape hatch independent of L2 sequencer cooperation.

L1 reorg — mitigated by finality waiting periods

If Ethereum L1 experiences a reorg, settled certificates could reference invalidated L1 state. The AggLayer mitigates this through GER Inclusion Proofs — each inserted Global Exit Root must have a valid Merkle proof inclusion in the L1 Info Root, ensuring proofs reference legitimate finalized L1 state. The `(agglayer-clock)` crate listens for L1 blocks to pace epochs, and the settlement client monitors transaction confirmation before marking certificates as settled. Ethereum's finality (2 epochs, ~12.8 minutes) provides the baseline safety window.

Prover bug — mitigated by audits and multiple prover implementations

A soundness bug in SP1, Plonky3, or the pessimistic proof logic itself could allow invalid proofs to verify on L1. This is the most critical residual risk. Mitigations include:

- **Multiple audits:** SP1 has been audited by Veridise, Cantina, Zellic, and KALOS
- **A real-world precedent was addressed:** A critical vulnerability in Plonky3's FRI verifier was discovered and patched in SP1 V5.0.0 before exploitation
- **Multi-zkVM optionality:** The `(Agglayer_PessimisticProof_Benchmark)` project benchmarks across SP1, Pico, RISC Zero, and OpenVM — enabling zkVM fallback if one proves unsound
- **Native execution testing:** Proof logic runs as standard Rust first (cheap, debuggable), then inside the zkVM (expensive, proven). Logic bugs are caught before proof generation

Governance capture — mitigated by chain isolation

The `(PolygonRollupManager)` on L1 is controlled by the `(PolygonAdminMultisig)`. Per L2BEAT's analysis, contracts are currently instantly upgradable with no exit window for users. This is a known risk across the L2 ecosystem — **86% of 129 tracked L2 projects** have no exit window (L2BEAT/arxiv analysis, 2025). The mitigation is architectural: even if governance is captured and a malicious upgrade is pushed, the pessimistic proof's per-chain balance isolation ensures that one compromised chain cannot drain the unified bridge. Timelock contracts exist on both L1 `(0xEf1462451C30Ea7aD8555386226059Fe837CA4EF)` and L2 `(0xBBa0935Fa93Eb23de7990b47F0D96a8f75766d13)` for phased upgrade deployment.

Key repository reference

Repository	Description	URL
AggLayer monorepo	Rust implementation: pessimistic proof, certificate orchestrator, epoch packer	github.com/agglayer/agglayer
AggLayer contracts	Solidity: PolygonZkEVMBridgeV2, RollupManager, GlobalExitRoot	github.com/agglayer/agglayer-contracts
Bridge-and-Call	BridgeExtension + JumpPoint for atomic cross-chain calls	github.com/agglayer/lxly-bridge-and-call
AggSandbox	Local development environment for AggLayer	github.com/agglayer/aggsandbox
SP1 zkVM	RISC-V ZK virtual machine (Plonky3 STARK backend)	github.com/succinctlabs/sp1
Plonky3	Modular STARK proving toolkit	github.com/succinctlabs/plonky3
lxly.js	JavaScript SDK for Unified Bridge interaction	github.com/0xPolygon/lxly.js
LAIR3-BDK	Blockchain Development Kit (CDK fork)	github.com/LAIR3
Kurtosis CDK	Upstream CDK deployment framework	github.com/0xPolygon/kurtosis-cdk
PP Benchmark	Pessimistic proof benchmarks across zkVMs	github.com/BrianSeong99/Agglayer_PessimisticProof

Repository	Description	URL
Unified Bridge explainer	Architecture documentation	github.com/BrianSeong99/AggLayer_UnifiedBridge

Conclusion

The cross-chain bridge problem is not a technology problem — it is a trust-architecture problem. **\$2.8 billion in bridge losses** trace to a single pattern: designs that concentrate trust in small committees, wrap tokens into exploitable honeypots, or rely on human honesty rather than mathematical proof.

The AggLayer Unified Bridge breaks this pattern through three innovations. First, a single unified escrow on Ethereum eliminates per-chain honeypots. Second, pessimistic proofs cryptographically enforce that no chain can withdraw more than it deposited, regardless of that chain's internal security. Third, hierarchical Sparse Merkle Trees — Local Exit Tree, Local Balance Tree, Nullifier Tree — provide the data structures that make per-chain accounting verifiable in zero knowledge.

openBDK inherits this foundation and extends it with a Relayer-Validator consensus layer for application-level customization. The security composition is deliberate: validators handle liveness and ordering; the pessimistic proof handles safety; Ethereum provides finality. A compromised openBDK validator set can harm its own chain's users but cannot drain the bridge, cannot affect other chains, and cannot forge deposits that didn't occur.

The most important insight from this research is not about any single bridge hack. It is about the failure of the industry to learn from the pattern. Ronin's 5-of-9 multisig was compromised in March 2022. Harmony's 2-of-5 multisig was compromised three months later. The same class of vulnerability, exploited by the same attacker (Lazarus Group), against two separate protocols. The AggLayer's pessimistic proof makes this class of attack architecturally impossible — not by adding more validators to the committee, but by **eliminating the committee from the security model entirely** and replacing it with cryptographic proof.

For developers building on openBDK: the Unified Bridge's `bridgeAsset`, `bridgeMessage`, `claimAsset`, and `claimMessage` functions are your cross-chain primitives. The Bridge-and-Call extension enables atomic cross-chain contract invocation. The code examples in this document compile against real interfaces deployed at real addresses on Ethereum mainnet. Build on them.